

CSC490 A4

Benson Li 1007815376
Yuchen Wang 1010441595
Xuanyi Lyu 1009343266
Zheyu Zheng 1008979580

November 8, 2025

Part One: Background research on GRPO

GRPO was first introduced in the paper DeepSeekMath: Pushing the Limits of Mathematical Reasoning in LLMs (arXiv:2402.03300), which also implemented GRPO to train the DeepSeekMath model. The paper successfully validated the effectiveness of GRPO by achieving new SOTA results among open-source small models across various math datasets.

The main improvement GRPO made upon PPO is that GRPO replaces the critic in PPO with a group-based relative baseline, significantly reducing training resources and shown great enhancements in the out-of-domain performance. Following this work, a series of papers generalized GRPO into the broader framework of Reinforcement Learning with Verifiable Rewards (RLVR), where an automatic verifier or judge provides binary or rubric-based rewards without human annotation. Below we summarize five of the most relevant RLVR papers to GRPO and our TweetVerify project, which aims to train an LLM-based judge to detect AI-generated political tweets.

Paper (short)	Year	Why relevant to Tweet-Verify	Short commentary / take-away
Reinforcement Learning with Verifiable Rewards Implicitly Incentivizes Correct Reasoning (Wen et al., arXiv:2506.14245)	2025	Provides the theoretical and empirical foundation for RLVR. It shows that verifiable binary rewards alone can teach models to reason more consistently.	Demonstrates that RLVR can outperform standard supervised fine-tuning even without human preference data. The paper's reward-verifier structure directly informs how we might design a correctness judge for tweet-authorship verification.

Paper (short)	Year	Why relevant to Tweet-Verify	Short commentary / take-away
Crossing the Reward Bridge: Expanding RL with Verifiable Rewards (Su et al., arXiv:2503.23829)	2025	Extends RLVR to broader natural-language and multi-domain tasks, exploring scalable verifier construction.	Highlights challenges of applying RLVR outside math or code, such as social text domains. Offers practical strategies for automatic verifier calibration—useful for TweetVerify’s real-world noisy data.
J1: Incentivizing Thinking in LLM-as-a-Judge (OpenAI Alignment Team, arXiv:2504.06781)	2025	Focuses on reinforcement learning for LLM-based judges that must deliberate before deciding.	Directly parallels our use case. J1’s “reason-then-decide” paradigm could improve judge interpretability and consistency when classifying tweets as human- or AI-written.
One Token to Fool LLM-as-a-Judge (Wei et al., arXiv:2505.08893)	2025	Analyzes adversarial vulnerabilities in LLM judges trained under RLVR setups.	Shows that even minimal prompt perturbations can flip judgments. Its attack taxonomy (trigger words, emoji injection, paraphrasing) will guide our adversarial robustness evaluation metrics.
Rubrics as Rewards: Reinforcement Learning Beyond Binary Correctness (Park et al., arXiv:2507.02156)	2025	Introduces multi-dimensional rubric-based verifiable rewards instead of simple pass/fail signals.	Suggests a richer reward structure for judging nuanced political content. In TweetVerify, similar rubric signals (plausibility, linguistic consistency, stance balance) could improve both accuracy and fairness.

Part Two: Metrics and Evaluations

Metric	Why We Care	Measurement (Quantitative & Qualitative)	Challenges at Scale
1. Classification Performance (Precision, Recall, F1-Score)	This is the fundamental measure of our model's correctness. High precision is critical to minimize false positives (wrongly accusing human authors of using AI), which could harm user trust. High recall is needed to maximize the detection of actual AI content, fulfilling our core objective. The F1-score provides a balanced measure.	Quantitative: Calculate Precision, Recall, and F1-score on a held-out, balanced test set. Track these scores per content category (e.g., news, casual chat). Qualitative: Manually review a sample of false positives and false negatives to identify systematic errors or patterns (e.g., does the model struggle with sarcastic human text?).	Obtaining a large, diverse, and perfectly balanced test set that reflects the ever-changing distribution of content on Twitter is a continuous challenge. Real-world data is inherently imbalanced.
2. Adversarial Robustness	AI-generated content producers will actively try to evade detection using techniques like paraphrasing, typo injection, or watermark removal. Our model is useless if it cannot withstand these attacks. This metric directly measures our system's resilience in an adversarial environment.	Quantitative: Measure the accuracy drop when the test set is perturbed by automated adversarial tools (e.g., QuillBot, DIPPER). Report the "attack success rate." Qualitative: Conduct red-teaming exercises where team members manually craft evasive tweets and analyze why the model failed.	The landscape of adversarial attacks evolves constantly. Creating a test suite that anticipates future obfuscation techniques is difficult and resource-intensive. Scaling manual red-teaming is impractical.
3. Generalization to Unseen LLMs	Our training data will inevitably be from a specific set of LLMs (e.g., GPT-4, LLaMA 3). However, new models are released constantly. We need to ensure TweetVerify remains effective against text from future, unseen LLMs to ensure its long-term viability.	Quantitative: After training on a set of LLMs, evaluate the model's zero-shot accuracy on a separate test set generated by a held-out LLM. Measure the performance degradation. Qualitative: Analyze the features the model relies on. Does it learn fundamental patterns of generated text, or does it overfit to the quirks of a specific model?	Acquiring and generating representative datasets from a wide variety of LLMs, especially closed-source or niche models, is costly and time-consuming. It's a constant race to keep the evaluation set current.

Metric	Why We Care	Measurement (Quantitative & Qualitative)	Challenges at Scale
4. Fairness and Bias	AI detection models can exhibit biases, for example, by being more likely to flag text written by non-native English speakers as AI-generated due to less complex sentence structures. We must ensure our tool is fair and does not disproportionately penalize certain user groups.	Quantitative: Use datasets annotated with author demographics (if ethically sourced) to measure performance disparities across groups (e.g., using metrics like Equalized Odds). Analyze false positive rates for different dialects. Qualitative: Solicit feedback from a diverse user base during pilot testing. Manually audit flagged content from various online communities.	Obtaining demographic data for social media users is fraught with privacy and ethical challenges. Proxies for demographics can be unreliable and introduce their own biases. Defining "fairness" across all linguistic and cultural contexts is a complex, unsolved problem.
5. Latency and Efficiency	TweetVerify must be practical for platform-scale deployment, capable of handling millions of tweets daily. Low latency is crucial for real-time applications (e.g., flagging content as it's posted) without disrupting the user experience.	Quantitative: Measure the average inference time per tweet (in milliseconds) and the maximum throughput (tweets per second) on target hardware. Qualitative: Assess the user-perceived speed during interactive testing. Is the system responsive enough for real-time use cases?	Laboratory performance benchmarks often don't capture real-world complexities like network overhead, variable batch sizes, and cold starts. Balancing model complexity (for accuracy and robustness) with computational efficiency is a difficult trade-off.

Table 2: Evaluation Metrics for the TweetVerify

Part Three: Establishing a baseline with a LLM

Task and Primary Metric

In this section, we establish a baseline for detecting AI-generated tweets using both a transformer-based large language model (LLM) and a simple supervised classifier. The task is formulated as a binary classification problem, where each tweet is labeled as either *Human* (0) or *AI* (1). We adopt the F1-score for the AI class as the primary evaluation metric, since it balances precision and recall and directly reflects the trade-off between incorrectly flagging human tweets as AI (false positives) and failing to detect AI-generated tweets (false negatives). In addition to F1, we report accuracy, precision, recall, and confusion matrices for both models to provide a comprehensive evaluation.

Dataset Construction and Splits

The dataset combines two preprocessed corpora: a set of high-quality human-written tweets and a set of AI-generated tweets, stored as `high_quality_human.csv` and `ai_generated.csv`. Each dataset is loaded into a `pandas` `DataFrame` and capped with a `max_samples` parameter to maintain balance and control runtime. The merged dataset is shuffled with a fixed random seed to avoid ordering bias.

We split the combined data into training, validation, and test sets using stratified sampling to preserve class balance. The default ratios are 70% training, 15% validation, and 15% test, ensuring representative sampling across subsets. Dataset statistics confirm that human and AI tweets are evenly distributed across all splits.

Transformer-based LLM Classifier

For the LLM baseline, we fine-tune a pretrained Hugging Face transformer model (`bert-base-uncased`) for binary text classification. Each tweet is tokenized and truncated to a maximum sequence length of 128 tokens, with the [CLS] embedding used for classification. The fine-tuning procedure employs the AdamW optimizer with a learning rate of 2×10^{-5} and early stopping based on validation F1-score.

Unlike zero-shot prompting, this approach directly learns discriminative boundaries from labeled data while leveraging the transformer’s contextual encoding ability. The model is trained for 3 epochs with a batch size of 16, achieving stable convergence without overfitting. The validation and test sets are reserved exclusively for model selection and final evaluation.

Logistic Regression Baseline on Embeddings

As a classical baseline, we train a logistic regression classifier using dense sentence embeddings derived from a pretrained SentenceTransformer (`all-MiniLM-L6-v2`). A `StandardScaler` normalizes embeddings before training a `LogisticRegression` model from `scikit-learn`.

This model represents a lightweight alternative that relies on fixed embeddings and a linear decision boundary. The validation set tunes the regularization strength, and the test set provides the final evaluation. While simple, this approach provides a transparent and computationally efficient baseline against which transformer-based methods can be compared.

Prompt-Only LLM Baseline

We evaluated a **prompt-only** large language model, `Qwen2.5-7B-Instruct`, to understand how well a general-purpose LLM can classify tweet authenticity without any gradient updates. The model was prompted with a binary question such as:

“Given the following tweet, decide whether it was written by a human or generated by AI. Answer with only one word: **Human** or **AI**.”

Each prediction was parsed and mapped to a binary label. The evaluation was run on the same balanced test set as the fine-tuned classifiers, with the following quantitative results.

Quantitative Comparison

We compare the logistic regression and transformer-based LLM on the same balanced dataset of 6,000 total training samples (3,000 per class) and 901 test tweets. Table 3 summarizes the results.

Metric	Logistic Regression	Prompt-only LLM	Transformer (BERT)
Accuracy	53.61%	52.89%	72.48%
F1-Score (AI class)	0.5486	0.6827	0.7243
Precision	0.5336	0.5185	0.7158
Recall	0.5644	0.9990	0.7331

Table 3: Performance comparison between logistic regression baseline, prompt_only llm baseline and fine-tuned transformer model.

The transformer achieves a significant improvement of nearly 20 percentage points in both accuracy and F1-score, indicating that contextual representations learned by BERT are far more effective at capturing stylistic and semantic nuances than static sentence embeddings.

The confusion matrices further illustrate this difference:

		Predicted Human	Predicted AI
2*Logistic Regression	Actual Human	229	222
	Actual AI	196	254
2*Transformer (BERT)	Actual Human	329	122
	Actual AI	126	324

Table 4: Confusion matrices for both classifiers on the test set.

The transformer correctly identifies 324 of 450 AI tweets and 329 of 451 human tweets, reflecting a more balanced trade-off between false positives and false negatives. The model generalizes better by leveraging bidirectional attention over contextual word patterns, idiomatic phrasing, and sentence rhythm.

1. Error Analysis and Common Mistakes

We analyze the types of errors made by each classifier to better understand their decision behaviour.

Logistic Regression. The logistic regression baseline exhibits several recurring failure modes:

- **Overly polished human tweets:** formally written human tweets were often misclassified as AI.
- **Informal AI tweets:** AI-generated tweets that mimic conversational tone were incorrectly labeled as human.
- **High URL/hashtag density:** tweets containing multiple URLs or hashtags confused the model, likely due to weak contextual encoding.

Transformer (BERT). The transformer model alleviates many of these issues but still struggles in specific cases:

- **Extremely short tweets** with minimal lexical content (e.g., “Amazing work!”) remain ambiguous.
- **AI paraphrases of human text** retain semantic coherence but lack distinctive stylistic features.
- **High creativity or irony:** figurative language and humor remain difficult for both models to interpret reliably.

Prompt-Only LLM (Qwen2.5-7B-Instruct). The prompt-only large language model exhibits a distinct set of systematic errors stemming from its instruction-following and alignment biases rather than data limitations. Its most common failure modes include:

- **Over-flagging bias:** The model shows a strong tendency to label nearly all tweets as “AI-generated”, even when the text is clearly human-written. This results from its alignment objective, which favors affirmative answers when asked to detect AI content.
- **Surface-level judgment:** It relies heavily on surface formality cues (e.g., clean grammar, consistent tone, or high lexical density) and equates them with AI authorship, ignoring contextual or semantic signals of genuine human spontaneity.
- **Context misinterpretation:** Short, ironic, or emotionally charged tweets are almost always predicted as AI because the model lacks situational grounding and misreads stylistic exaggeration as artificiality.
- **Lack of calibration:** Unlike fine-tuned classifiers, the model has no learned decision boundary or threshold control, leading to extreme recall (1.0) and weak precision (0.52).

Quantitatively, the transformer’s improvement in recall (0.7331 vs. 0.5644) shows that it better identifies AI content, though at a modest cost in false positives. Qualitative inspection suggests that the model’s contextual attention helps it capture subtle stylistic markers such as unnatural repetition or overly consistent sentence rhythm that static embeddings miss.

However, when contrasted with larger prompt-only LLMs such as **Qwen2.5-7B-Instruct**, we observe a different limitation: while BERT learns explicit discriminative boundaries through supervision, the zero-shot LLM understands the concept of AI generation but fails to apply consistent evidential criteria.

2. Limitations and Next Steps

All three models present complementary limitations. The logistic regression classifier is interpretable and lightweight but fails to model non-linear relationships or contextual dependencies between tokens. The transformer-based model, while substantially more accurate, is computationally expensive and prone to overfitting on stylistic artifacts specific to the training data. The prompt-only Qwen2.5-7B-Instruct model, despite showing some zero-shot capability for authorship detection, lacks gradient-based adaptation and cannot learn task-specific decision boundaries. Its outputs are dominated by prior alignment biases rather than discriminative evidence, leading to

nearly perfect recall but poor precision—treating “AI detection” as a one-sided verification problem rather than a balanced classification task.

This comparison highlights that none of these architectures alone achieves an ideal precision–recall balance: supervised fine-tuning provides discrimination but limited generalization, whereas prompting yields broad reasoning ability but poor calibration. These observations motivate our transition in Part 4 to SFT followed by GRPO to unify both strengths—explicit discriminative training and reinforcement alignment.

In future work, we plan to:

1. Expand the LLM experiments to larger architectures (e.g., `Qwen`, `roberta-base`, `DeBERTa-v3`) and conduct ablation studies on tokenization length and layer freezing.
2. Incorporate metadata-driven features into multimodal baselines for richer contextual cues such as timestamps, author handles, and posting devices.
3. Perform few-shot or in-context learning experiments with generative models to explore hybrid reasoning approaches that combine interpretability and adaptability.

Together, these directions aim to develop a more robust and generalizable AI-text detection system that balances interpretability, efficiency, and real-world reliability.

Part Four: Training your Judge

Metric and Hypotheses

Our reinforcement learning objective is to directly optimize the **F1-score** of the AI detection classifier. The F1 metric is verifiable, label-based, and reflects the trade-off between recall and precision that is critical for tweet-authorship detection. We hypothesize that Reinforcement Learning with Verifiable Rewards (RLVR) through Group Relative Policy Optimization (GRPO) will yield a model that reduces the over-recall bias of the prompt-only LLM while preserving its strong coverage of AI content.

Training Pipeline Design

We implemented a two-stage RLVR pipeline orchestrated entirely within the **Modal** cloud runtime:

1. **Supervised Fine-Tuning (SFT)**: Uses `Qwen2.5-14B-Instruct` as the base model and applies LoRA adapters. SFT trains on balanced human/AI tweet datasets (15k each) using AdamW optimizer with linear warmup scheduling, early-stopping on validation F1, and automatic logging to TensorBoard/W&B. Once $F1 \geq 0.73$ and precision ≥ 0.70 for two consecutive evaluations, the Modal API automatically triggers the GRPO phase.
2. **Group Relative Policy Optimization (GRPO)**: Initializes from the best SFT checkpoint and fine-tunes the model via reinforcement learning on verifiable F1 rewards. The policy model and reference model are both loaded in multi-GPU mode manually. The GRPO run is fully self-contained, with real-time metric logging, checkpoint commits, and confusion matrix visualization.

Each phase maintains persistent volumes under `/mnt/cache/`, and automatic checkpoint management functions (`list`, `rollback`) ensure reproducibility and rollback safety.

Base Model and Optimizer

The base model is **Qwen2.5-14B-Instruct**, initialized in mixed-precision (`bfloat16`) and distributed across 8×A100-40GB GPUs. We employ the **AdamW** optimizer in both stages with weight decay = 0.01. A linear warmup scheduler (`get_linear_schedule_with_warmup`) controls the effective learning rate.

The key hyperparameters for SFT are:

- Learning rate: initially 5×10^{-5} , reduced to 5×10^{-6} for stability
- Batch size: 8 (gradient accumulation = 4)
- Epochs: 4, early stopping after 2 stagnant validation epochs
- LoRA: $r = 16$, $\alpha = 32$, dropout = 0.05

Preliminary experiments with higher learning rates (5×10^{-5}) showed unstable gradient norms and oscillating F1, whereas 5×10^{-6} produced smoother and reproducible convergence.

RL Algorithm and Reward Design

We adopt Group Relative Policy Optimization which computes relative advantages within groups instead of relying on a critic. The reward combines individual correctness, model confidence, and group-level F1 improvement:

$$R_i = 1.0 \cdot \text{Correct}_i + 0.1 \cdot (p_i - 0.5) + 0.5 \cdot (F1_{\text{group}} - 0.5)$$

and the loss is given by:

$$\mathcal{L} = -E[(R_i - \bar{R}) \log \pi(a_i | s_i)] + \beta KL(\pi || \pi_{ref})$$

where $\beta = 0.05$ is the KL penalty balancing exploration and stability. This formulation enables direct optimization of verifiable F1 metrics without training an explicit reward model.

Logging and Checkpoint Management

Both SFT and GRPO stages record detailed metrics through:

- **TensorBoard**: loss curves, precision, recall, F1 trends per epoch (saved in files in modal’s storage).
- **Weights & Biases**: synchronized logging of hyperparameters, reward terms, and confusion matrices.
- **Modal Volumes**: persistent directories under `/mnt/cache/` for automatic checkpoint commits (e.g., `sft_run_20251107_*`, `grpo_run_20251107_*`).

Utility functions `list_checkpoints()` and `rollback_checkpoint()` support reproducibility and controlled rollback, automatically updating a symbolic link to `latest_active` snapshots.

Ablations and Observations

We conducted targeted ablations to assess sensitivity:

- **Learning rate**: 5×10^{-5} caused divergence and NaNs in F1; lowering to 5×10^{-6} stabilized training.

- **Early Stop:** We write early stop = 3, but it make the training stop at the first run sometimes, so we skip early stop for the first 2 epoch and now it runs properly.
- **Group size:** Small groups (≤ 4) introduced reward noise; group size 16 offered best gradient stability.

What Didn't Work and Next Steps

Initial GRPO trials exhibited excessive KL divergence and recall collapse when using high learning rates or poorly balanced reward weights. We revised the reward coefficients and scheduler to mitigate instability. Although full GRPO convergence results are pending, early experiments show that the pipeline executes end-to-end reliably on 8 GPUs and produces valid checkpoints.

In subsequent work, we plan to:

1. Complete multi-epoch GRPO runs and evaluate F1 gains over the SFT baseline.
2. Add **more verifiable rewards** to improve the reliability and transparency of the training feedback signal.
3. Explore **rubric-based rewards** (e.g., coherence and stance neutrality).